how-git token and complexity audit

Target: `/Users/waylon/local_stage/how-git`, branch `dev` at `5028b586` (2026-07-19, `[MERGE] pool/v2026.0/n164.feature.user-skill-overrides into dev: v2026.0.136`), read-only via `git show dev:<path>`. Auditor: Claude Fable 5.1, 2026-10-29. Every claim is tagged VERIFIED (command or quote given) or UNVERIFIED.

Token method: tokens = bytes / 3.6 for the markdown surfaces (stated heuristic; the Opus 4.7+ tokenizer shared by Opus 5.5 and Fable 5.1 is 1x to 1.35x denser than older tokenizers per the claude-api skill, so the real count is likely 3.6 to 4.9 per byte: treat every figure below as a floor). Where a transcript receipt exists the measured figure is used instead and marked so. Read-tool output adds a `NNN\t` line-number prefix per line (about 2 tokens per line); the mandated set has 2,718 lines, so add about 5,400 tokens to the raw-byte estimate when the files are loaded through Read.

0. Prices and models (VERIFIED by the research subagent, URLs quoted)

Model	Input	5m cache write	Cache read	Output	Source
Claude Fable 5.1 `claude-fable-5-1`	$10/MTok	$12.50/MTok	$0.25/MTok	$50/MTok	https://platform.claude.com/docs/en/about-claude/pricing: "Claude Fable 5.1 | $10 / MTok | $12.50 / MTok | $20 / MTok | $0.25 / MTok | $50 / MTok"
Claude Opus 5.5 `claude-opus-5-5`	$4/MTok	$5/MTok	$0.20/MTok	$20/MTok	same page: "Claude Opus 5.5 | $4 / MTok | $5 / MTok | $8 / MTok | $0.20 / MTok | $20 / MTok"
OpenAI GPT-6 Astra `gpt-6-astra`	$10/MTok	$12.50/MTok (1.25x)	$1.00/MTok	$50/MTok	https://developers.openai.com/api/docs/pricing.md: "gpt-6-astra | $10.00 | $50.00 | $1.00 | $12.50" and https://developers.openai.com/api/docs/models/gpt-6-astra.md: "1,050,000 context window", "Prompts with more than 272K input tokens are priced at 2x input and cache rates"

Astra identity: "Astra" resolves to OpenAI's GPT-6 Astra (public API, specs above, VERIFIED). Google's Project Astra is a research prototype with no API or pricing (VERIFIED: deepmind.google/models/project-astra "Project Astra is a research prototype"). No Anthropic, xAI, Meta, Mistral, DeepSeek, Alibaba or coding-agent product named Astra surfaced (negative search finding, not exhaustive). Analysis below uses GPT-6 Astra's published numbers; its instruction-following behaviour is UNVERIFIED and treated as a generic third-party agent.

Cache rules (VERIFIED, https://platform.claude.com/docs/en/build-with-claude/prompt-caching): "By default, the cache has a 5-minute lifetime. The cache is refreshed for no additional cost each time the cached content is used." Claude Code (VERIFIED, https://code.claude.com/docs/en/prompt-caching): main conversation gets "One hour" TTL on a Claude subscription, "Five minutes" on API key or usage credits; "Each request that hits the cache resets the timer". Consequence for this audit: every file a session reads stays in the conversation and is re-sent on every later turn, billed at the cache-read rate while the cache is warm and at the full input rate after a gap longer than the TTL. On an API key a single compile or test run longer than five minutes makes the next turn re-bill the whole git context at $10/MTok (Fable) or $4/MTok (Opus 5.5).

1. The token-relevant surface (VERIFIED: `git ls-tree -r -l dev`, `wc -c`)

What the loader `/Users/waylon/.claude/skills/how-git/SKILL.md` step 2 orders on every invocation: "read every `<repo>/.io.v4rian.how-git/skills/git-*/SKILL.md`" plus "`docs/Git_Strategy.md` with the ceremonies in `docs/RUNBOOK.md`; read them directly, never answer from a paraphrase" (loader line 13), plus four memories (line 12).

Surface	Bytes	Lines	Tokens (B/3.6)
Loader `how-git/SKILL.md` (injected by the Skill tool)	11,532	85	3,200
4 memories named in step 1	3,754		1,040
13 `git-*` skills (sum)	147,364	2,296	40,900
of which `git-shuttle`	42,402	510	11,800
of which `git-runtime-audit`	13,246	269	3,700
of which `git-commit-audit`	12,727	156	3,500
of which `git-merge-audit`	11,719	165	3,250
of which `git-strategy-audit`	11,695	189	3,250
`docs/Git_Strategy.md`	22,302	428	6,200
`docs/RUNBOOK.md`	49,222	490	13,700
Mandated per invocation	**234,174**	**2,718**	**65,000 (+5,400 Read prefixes = ~70,000)**
Also loaded by the loader's inventory table: in-repo `how-dev`	5,636	78	1,560
Not mandated but referenced as authoritative: `README.md` 34,648, `REQUIREMENTS.md` 65,594, `docs/CHANGELOG.md` 52,112	152,354		42,300
Always-on per session (independent of how-git): `~/.claude/CLAUDE.md` 8,259 + `MEMORY.md` 4,629	12,888		3,600

Deterministic enforcement that exists regardless of what the model reads (VERIFIED by reading each file):

- `~/.claude/hooks/guard-destructive.sh` (PreToolUse on Bash, wired in `~/.claude/settings.json`): denies `rm` outside temp roots, destructive git without a fresh `preflight-bak-*` ref plus bak zip under 12 h, push to main, bare `--force`, `branch -D`, `rebase`, `filter-branch`, `filter-repo`, `stash drop`, `reflog expire`, `git add -A|.`, staging banned files or `FEATURE_REQUESTS.md`, `Co-Authored-By` trailers, `filter-branch -- --all`, remote repo deletion, credential echo, sudo/brew/npm -g/gcloud.
- `~/.claude/hooks/git-preflight.sh`: the preflight ref plus zip.
- In-repo githooks: `pre-commit` (em/en dash scrub of staged files), `prepare-commit-msg` (auto-fix library: drops Co-Authored-By, scrubs dashes, replaces banned word "plus", derives `[TAG]` from the diff, caps the description at 3 sentences, rewrites bullets to third person, then `audit_message` aborts on a missing title or description), `commit-msg` (8-100 char subject, 140 for `[MERGE]`/`[REQ]`), `pre-push` (branch-name regex `^(pool|safety)/(vYYYY.N/)?nN.(feature|fix|r&d|refactor|reconcile|doc)...`), `post-merge` (off-grid claim on dev/main).
- Server side: Repository Rules (`rulesets/*.json`), the allocator workflow, and `ci-dispatch.yml` running `ci/audits/push/required.{commit-audit,compile,coverage,runtime}.sh` and `required.offline-test-suites.sh` as required status checks. `required.commit-audit` runs the deterministic em-dash gate and then `claude_judge_commit_range` with the full `git-commit-audit/SKILL.md` body as the prompt prefix, once per push (VERIFIED: `scripts/_lib.sh` claude_judge_commit_range: `printf 'LENS BODY (treat as authoritative):\n%s\n\n' "$skill_body"`).

So the prose a model reads is not the enforcement layer; Git_Strategy.md says so itself (VERIFIED, lines 385-391: "Hooks are guardrails, CI is authoritative ... nothing developer-side can be the enforcement layer").

2. Findings: where tokens go and why

2.1 The fan-out is unconditional (biggest single cost)

Loader line 13 mandates all thirteen `git-*` skills plus both docs for every task, including a one-line commit. The loader's own decision tree (lines 63-77) names the one to three skills each task needs; the two instructions coexist, and the earlier, stronger one ("read every") wins for a literal reader. VERIFIED (quoted above). Cost: about 65,000 tokens entering context once per invocation, then re-sent on every subsequent turn of the session.

Per-task need versus mandated load (bytes, B/3.6):

Task	Skills the decision tree names	Needed bytes	Needed tokens	Mandated tokens	Waste
Compose a commit	`git-commit-audit` + identity memory	12,727 + 1,345	3,900	65,000	94%
Create a working branch	`git-strategy-audit` (or `bin/create-pool-branch --help`)	11,695	3,250	65,000	95%
Merge working branch to dev	`git-merge-audit` + RUNBOOK "Land a feature" (lines 122-144, ~2 KB)	13,700	3,800	65,000	94%
Merge dev to main	`git-merge-audit`, `git-runtime-audit`, RUNBOOK "Ship a release" (lines 145-180)	27,000	7,500	65,000	88%
History rewrite (mine only)	`how-preflight`, `git-history-preservation`, `git-history-rewrite`	18,107	5,000	65,000	92%
Snapshot rebuild from zips	`git-snapshots` + every sub-skill	~110,000	30,500	65,000	53%
Rebuild from working-tree folders	`git-shuttle` + sub-skills	~115,000	32,000	65,000	51%

2.2 Duplicated rules (same rule, three to five copies loaded in one session)

VERIFIED by grep; file:line given.

Rule	Copies loaded per session	Deterministic owner
No `Co-Authored-By` trailer	CLAUDE.md:43; loader implicit; `git-commit-audit`:42 (rule 17); `git-shuttle`:366, 474; `smart-commit` prompt	`guard-destructive.sh` deny + `auto-fix-commit-message.sh` step 1
No `git add -A` / `.`	CLAUDE.md:44; `git-commit-audit`:68 (rule 30); `git-shuttle`:357, 473; `git-snapshot-preflight` (python uses `add -A` on a scratch clone, contradicting it)	`guard-destructive.sh`
Banned staged files list	CLAUDE.md banned list; `git-commit-audit`:66 (rule 28); `git-snapshot-preflight`:28-31; `git-coverage-audit`:18-24; local_stage CLAUDE.md "Workspace-wide banned files"	`guard-destructive.sh` BANNED_STAGE_RE
Em/en dash ban	`git-commit-audit`:32, 40 (rules 10, 15); `git-merge-audit`:21 (rule 4); loader:34; `git-shuttle`:365; RUNBOOK:446-453	`pre-commit` scrub, `prepare-commit-msg` scrub, CI em-dash gate in `commit-audit.sh`
Approved `[TAG]` set	`git-commit-audit`:22; `git-history-preservation`:27; `auto-fix` _AUTO_FIX_TAGS	`prepare-commit-msg` derives and validates
Subject length	`git-commit-audit`:30 (72 chars); `git-merge-audit`:20 (72 chars)	`commit-msg` hook (8-100, 140 for MERGE/REQ): the prose and the hook disagree (72 vs 100)
`--no-ff` merges into dev	loader:22; `git-commit-audit`:101 (rule 38); `git-strategy-audit`:111 (rule 8); `git-merge-audit`:92-119; Git_Strategy.md:20, 26	`bin/version` counts first-parent merges; `claim-dev-merge.sh`; `init-pool-merge`
Never pick nN yourself / never push pool/* directly	loader:33; `git-commit-audit`:99; `git-strategy-audit`:36, 85, 176-177; Git_Strategy.md:163-220	allocator + `pool-creations.json` ruleset + `pre-push` note
Never trash safety anchors	loader:31; `git-strategy-audit`:37; Git_Strategy.md:114-159, 377-383	`safety-lockdown.json` ruleset
Preflight before destructive ops	CLAUDE.md:18 (rule 12); `how-preflight` skill; `git-history-rewrite`:33, 143; `git-snapshot-preflight` whole file; `git-snapshots`:48-63	`guard-destructive.sh` + `git-preflight.sh`

Rule	Copies loaded per session	Deterministic owner
User-authored commits read-only	CLAUDE.md:17 (rule 11); `git-history-preservation` whole file; `git-history-rewrite`:10; `git-shuttle`:472; `git-snapshots`:148	none (judgment); one copy is enough
Commit identity by org	memory `feedback_commit_identity_by_repo_org`; loader:36; `git-history-rewrite`:12 (contradicts: "reads identity from git config"); `git-snapshot-preflight`:16 (same contradiction)	none; the memory is the single statement, the two skills contradict it

Estimated duplicated prose: about 18 KB across the mandated set (5,000 tokens), plus the contradictions below which cost more than their bytes because they force re-reading or a question.

2.3 Contradictions (VERIFIED, file:line)

1. **Canonical path.** Loader:13 "read the canonical copy at `/Users/waylon/local_stage/fractaris/.io.v4rian.how-git/skills/`"; loader:81 "the latest spec is the local canonical `local_stage/how-git` dev, not GitHub". The fractaris copy of `git-commit-audit` differs from dev by 97 diff lines (VERIFIED: `git show dev:...git-commit-audit/SKILL.md | diff - fractaris/...` = 97 lines; the other 13 are identical). `~/.claude/hooks/skill-lint.sh` also points FW at fractaris.
2. **Single commit straight to dev.** `git-commit-audit`:18 (A.5) and :97 (G.36), `git-strategy-audit`:58, 145, 172: "1 atomic commit → straight to dev". Loader:22-24 and Git_Strategy.md:46-60: dev takes no direct commits, every change rides a pool branch. Under the pre-push hook and the rulesets a direct commit to dev is only possible off-grid; the skills still teach it as the default.
3. **Version source.** `git-commit-audit`:107, `git-merge-audit`:23, 138-139: "`cat VERSION`". Git_Strategy.md:369-375: "Tags are the seal (no VERSION file) ... The file model was tried and scrapped (HG-S02)"; `git-merge-audit`:163 itself says "no VERSION file". `git-strategy-audit`:19 still lists "`VERSION` file with `vYYYY.main.dev` shape" as a model marker.
4. **Merge body shape.** `git-commit-audit`:39 (rule 14) and :109 (rule 41): "[MERGE] body IS the description paragraph only - no bullets". `git-merge-audit`:28 (rule 5) "The body IS the description paragraph. 1-5 sentences" then :29 (rule 6) "Use a NESTED list breakdown ... every list line ends with a period"; :140 pre-push lens "Body parses as one paragraph with no leading bullets". Loader:29 "Merge body: nested list". Three incompatible shapes in the mandated set; the CI judge receives the commit-audit copy.
5. **Branch naming inside skills.** `git-merge-audit`:105-110 and `git-commit-audit`:18 use `feature/<short_name>`; `git-shuttle`:291, 322 "feature/"; v4rian is `pool/nN.*`. `git-shuttle`:318 introduces a `[CLEANUP]` tag absent from the approved set; :364 "`Changes:` always" versus `git-commit-audit`:38 "No `Changes:` header".
6. **Identity.** `git-history-rewrite`:12 and `git-snapshot-preflight`:16, `git-snapshots`:20: "read from git config user.name, never hardcode" versus memory `feedback_commit_identity_by_repo_org` ("never the machine's global git identity") and loader:36.
7. **Two skills named `how-dev`.** In-repo `how-dev` (5,636 B, "BINDING ... watch every CI run") versus `~/.claude/skills/how-dev` (surgical-minimal discipline) and `how-push` ("Default ... remote CI is not part of the definition of done ... never propose CI"). Loader:13 "in-repo wins" makes the CI-watch rule load inside every v4rian repo, where how-push says the opposite.
8. **Timezone.** `git-shuttle`:172, 330-335 "LA timezone always", `git-commit-audit`:81-85 UTC-from-epoch with "ASK ONCE" on ambiguity; `git-snapshot-preflight`:117 UTC `Z`.
9. **Workflows that do not exist.** `git-commit-audit`:8, `git-strategy-audit`:8, `git-coverage-audit`:8, `git-compile-audit`:8, `git-runtime-audit`:8: ".github/workflows/audit-pushed-branch.yml"; `git-merge-audit`:126 and `git-commit-audit`:111: "dev-merge.yml / main-merge.yml". VERIFIED absent: `git ls-tree dev .github/workflows/` lists only allocate-pool-branch, bootstrap-ci, canaries, ci-dispatch, doc-freshness, seed-dev-on-main-create, validate-pr-open. RUNBOOK:446 also lists `build-on-tag.yml`, absent.
10. **Tools that do not exist.** `git-shuttle`:232 "Run `tools/content-match.sh` (shipped with this skill)", :252 `tools/integration-check.sh`. VERIFIED: `git ls-tree -r dev .io.v4rian.how-git/skills/` contains only SKILL.md files.

2.4 Dead memory links

41 distinct `[[wiki_link]]` targets, about 70 occurrences, across the skills (VERIFIED: `git grep -o -h -E '\[\[[a-z_0-9]+\]\]' dev -- .io.v4rian.how-git/skills | sort | uniq -c`). None of them exists in `/Users/waylon/.claude/projects/-Users-waylon-local-stage/memory/` except `feedback_user_authors_when_rebuilding_history` (VERIFIED: `ls` of the memory dir). Examples: `[[reference_v4rian_branch_creation_endpoint]]` x4, `[[feedback_no_developer_branch_creation]]` x4, `[[feedback_sym9_never_touches_dev_or_main]]` x4, `[[reference_v4rian_versioning]]` x3, `[[workflow_git_commits]]`, `[[reference_voice_ask_skill]]`, `[[bin]]`. Each is an instruction to go read something that is not there; a literal model spends a tool call (Read fails, ~~300-600 tokens round trip) or a grep per link.~~ ~~`skill-lint.sh` catches dead links only under `~~/.claude/skills`, not in the vendored tree.

2.5 Prose that a script already enforces (redundant reading)

- `git-commit-audit` sections B.6-B.10 (tag set, length, dashes), C.15-17, E.28-31: enforced by `prepare-commit-msg`, `commit-msg`, `pre-commit`, `guard-destructive.sh`. About 2.5 KB.
- `git-strategy-audit` regex and naming sections (lines 77-107) and the "Banned shortcuts" block: enforced by `pre-push`, the allocator and `pool-creations.json`. About 3 KB.
- `git-merge-audit` "Strategy" 14-17 and date mechanics 11-13: `init-pool-merge/finalize-pool-merge/bin/ship/claim-*.sh` own the merge, tag and date. About 2.5 KB.
- `git-runtime-audit` and `git-compile-audit` detailed shell recipes (lines 85-200 and 14-73): `ci/audits/push/required.runtime.sh`, `required.compile.sh` and the workspace `compile` driver run these; the skill's hand-rolled `cmake`/`open`/`pkill` recipes contradict `how-compile` ("never hand-run cmake"). About 12 KB.
- `git-snapshot-preflight` python builder (lines 45-131): `~/.claude/hooks/git-preflight.sh` is the tool the workspace actually uses. About 4 KB.

Roughly 24 KB (6,700 tokens) of the mandated 234 KB is recipe text for things a hook or script does.

2.6 Judgment-shaped rules that could be deterministic

- Merge body "macro tone check" grep list (`git-merge-audit`:141-142): already written as regexes; belongs in `auto-fix-commit-message.sh`'s `audit_message` for `[MERGE]` subjects, with the skill keeping one line.
- `V.10` symbol existence (`git-commit-audit`:55-62): a `scripts/audits/symbols.sh` that extracts backticked identifiers from the body and greps the committed tree is deterministic; the CI judge then only rules on semantics.
- Date monotonicity and author==committer date (`git-commit-audit`:87-90): one `git log --format` check.
- Allocator numbering drift (loader:33, 83): `create-pool-branch` should run `git ls-remote` itself; `pool/v2026.0/n165` on this repo does exactly that for clone refs (see section 6).
- "Confirm in one line: git rules loaded (memories + skills ...)" (loader:15): forces a count and a sentence every invocation; the Skill tool invocation itself is the receipt.

2.7 Ceremonies that cause retry loops

(Transcript receipts in section 3.) Structural causes visible in the code:

- `guard-destructive.sh` denies a destructive git command until a preflight exists; the deny text carries the exact fix, so the loop is bounded at one retry, but each denial costs the original tool call plus the deny JSON plus the preflight run plus the retry.
- `commit-msg` 100-char cap versus the skill's 72-char guidance: no loop, the stricter prose wins.
- `post-merge` claim refuses a dirty tree (loader:37 documents the workaround): a fully documented loop that a `git stash`-free `bin/finalize-pool-merge --allow-untracked` would remove.
- Off-grid allocator numbering restarts after pruning (loader:83, pool n165 fixes it): recreate-the-pair loops.

2.8 Model-specific failure modes

- **Fable 5.1 and Opus 5.5 (over-prescription).** The claude-api skill's migration notes for Fable state: "prompts written for prior models are often too prescriptive and reduce output quality" (VERIFIED, skill text loaded this session). The mandated set is 2,700 lines of imperative rules, many obsolete; the model either follows an obsolete one literally (`cat VERSION`, `feature/<name>`, "straight to dev") or spends reasoning reconciling contradictions. Opus 5.5 defaults to `effort: medium` (VERIFIED, same skill): less reasoning available to resolve the conflicts, so literal compliance with the first rule read is the likely outcome.
- **Long-context dilution.** 65k tokens of rules at the top of a session that then runs a 200k-token build or rebuild pushes Claude Code toward auto-compaction ("auto-compaction, which summarizes conversation history when approaching context limits", VERIFIED code.claude.com/docs/en/costs). After compaction the rules are gone and the loader is re-invoked: the fan-out is paid twice. Section 3 counts re-reads.
- **GPT-6 Astra.** Cached input is $1.00/MTok, four times Fable's $0.25 and five times Opus 5.5's $0.20, so re-sending 65k tokens per turn costs $0.065 per turn on Astra versus $0.016 on Fable; over a 60-turn session that is $3.90 versus $0.98. Its ">272K input priced at 2x" cliff (VERIFIED quote above) makes a 65k preload a quarter of the budget before the task starts. Its handling of contradictory instruction sets is UNVERIFIED.
- **Over-literal compliance (all three).** Loader:13 "never answer from a paraphrase" plus "read them directly" forbids the obvious optimisation (use the decision tree, read one skill); loader:19 "Every audit on every commit, push, PR and branch, compile and runtime audits included" read literally means running `git-runtime-audit` prose recipes for a docs commit.

3. Real usage evidence (transcripts)

Source: 226 `.jsonl` transcripts under `/Users/waylon/.claude/projects/-Users-waylon-local-stage/` (19 main sessions, 206 subagent sidechains, 1 `.bak`), 708 MB, 183,527 lines, 2026-02-27 to 2026-10-29; every other project directory (107 of 108) is empty, so the measurable window is effectively October 2026. Mining script and full tables: `/private/tmp/claude-502/-Users-waylon-local-stage/e38c8e77-4713-42b5-a7f6-6286f7af57c3/scratchpad/mine/results.md` (VERIFIED, `python3 -I mine.py` over the files; usage taken from `message.usage`, deduplicated by `message.id`).

3.1 Frequency (VERIFIED, Bash tool_use commands, October 2026)

Action	Count	In sessions that touched how-git
`git commit`	707	698
`git merge`	269	262
`git tag`	135	126
`git push`	260	253
`create-pool-branch`	251	241
`claim-dev-merge`	76	75
`ship` (word match, noisy)	26	26
`git-preflight.sh`	73	71
`git rebase`	48	44
`filter-repo` / `filter-branch`	8	8
`smart-commit`	1	1

`Skill` invocations: `how-git` 32, `how-dev` 2, `how-preflight` 1 (Tier A: 7 main + 25 subagent sessions in 8 parent trees; one tree, `f9e94080`, made 22 of the 32 calls through its subagents). None of the 13 `git-*` skills was ever invoked through the Skill tool; they are read as files. Sessions with any how-git reference (Tier B, includes work *on* how-git such as ai-reaudit and reconstruction): 100 of 226. `smart-commit`, the tool built so the parent "does NOT need to load the audit skills" (VERIFIED, `bin/smart-commit` header), appears 7 times against 714 commits (counts drifted by a few units between the two mining passes because sessions were still being appended; the table above is the first pass).

3.2 What actually gets read (VERIFIED, section 3 of results.md)

Across the 100 Tier-B sessions: distinct how-git surface files read, mean 2, median 1, max 17; surface reads mean 3, median 2, max 23; re-reads of the same path mean 1, median 0, max 11; bytes of surface text returned mean 22,793, median 12,744, max 319,045. Restricted to the 32 Tier-A sessions (a real `Skill how-git` call): distinct files mean 2, median 1, max 7; re-reads mean 0, max 3; most-read files `git-commit-audit/SKILL.md` x36, `git-merge-audit/SKILL.md` x15, `docs/RUNBOOK.md` x5, `Git_Strategy.md` x3; loader step 1 (read the four memories) executed in 7 of 32 sessions, step 2 (read vendored skills) in 29 of 32. **Direct ceremony load per Tier-A session (skill text + surface reads + git memories): 23.1 KB mean, about 5,800 to 6,400 tokens (median 6,000, max 13,800); summed over the month 739 KB, about 185,000 tokens, 0.18 percent of all new tokens in the corpus.** The `how-git` Skill injection itself: 32 injections, mean 10,335 bytes, max 11,354 (331 KB total, 45 percent of the direct load).

So the "read every git-* skill plus both docs" mandate (234 KB) is **not followed in practice**: the median session reads one skill (`git-commit-audit` is the most-read file in the receipts, often via `sed -n/head` slices of 1-13 KB) and the mean reads about 23 KB, one tenth of the mandate. Receipts: `f9e94080.../agent-ab51179b4e8b5d109.jsonl:47` reads `git-commit-audit/SKILL.md` (10,254 bytes) twice; `agent-ac38fe98ea17320a4.jsonl:27,36` reads it as two slices (7,362 then 12,291 bytes); `agent-af4da567f34d01aa9.jsonl:53-71` is the one session that read the wider set (README, Git_Strategy, four skills, the loader: about 83 KB). The outlier at 319,045 bytes is an audit session reading the framework itself, not a git task.

Two conclusions follow. First, the rules in the eleven skills a median session never opens cannot be what keeps history clean; the hooks, rulesets and claim scripts are, which is the macro's own position. Second, the loader's step 2 and its "Confirm in one line: git rules loaded (memories + skills ...)" produce a confirmation the model cannot honestly give (UNVERIFIED whether the printed counts matched the reads; not sampled), and a literal-compliance model (Opus 5.5 at default `medium` effort, or a third-party agent) would pay the full 65,000 tokens the Claude sessions silently skipped.

3.3 Measured token cost (VERIFIED numbers, interpretation stated)

- New tokens on the assistant turn that ingests the `how-git` Skill text: n=32, mean 6,337, median 5,378, min 4,195, max 14,633. The skill body is 10,335 bytes mean, so the loader costs between 2,900 tokens (bytes/3.6) and about 5,400 tokens (the whole measured turn); the markdown-with-paths tokenizes closer to 2 to 3 bytes per token than 3.6. **Calibration consequence: every bytes/3.6 figure in this report is a floor; multiply by 1.3 to 1.8 for a likely value.**
- 11-turn window from the first how-git trigger: new tokens mean 58,362, median 53,164 (n=100); the all-sessions baseline for non-overlapping 11-turn windows is mean 42,415, median 16,968 (n=2,183). The git ceremony window carries about 16,000 more new tokens than an average window and three times the median (upper bound: unrelated work in the same turns is included).
- Anchored on the first `git commit`: sessions with how-git loaded, window new tokens mean 18,480, median 12,711 (n=65); the three sessions that committed without any how-git trigger: mean 14,349 (n=3, too few to be more than indicative). Difference about 4,000 tokens per commit ceremony.
- Context occupancy, the dominant cost: window `cache_read_input_tokens` mean 999,798 over 11 turns, so the average turn re-reads about 91,000 cached tokens; Tier-A session totals mean 960 K new tokens (median 381 K) and 95 M cache-read tokens. The ~6,000 tokens of git prose a Tier-A session loads are re-read on about 258 later turns per session (mean), so about 1.5 M cache-read tokens per session are attributable to the ceremony (upper bound; the miner's stricter attribution gives 188 K as the floor). That is 6 to 7 percent of the average turn's 91 K context. Per session: $0.39 (Fable, $0.25/MTok), $0.31 (Opus 5.5), $1.55 (Astra); per month over 32 sessions: about $12 / $10 / $48 occupancy plus writes of 185 K tokens at cache-write price: $2.3 / $0.9 / $2.3. **Practised monthly cost of the ceremony text: roughly $15 (Fable), $11 (Opus 5.5), $50 (Astra).**
- If the loader were obeyed (65,000 tokens per invocation, 32 invocations, about 258 later turns per session), occupancy alone would be 32 x 65,000 x 258 = 537 M cache-read tokens per month: $134 (Fable), $107 (Opus 5.5), $537 (Astra), plus the writes 2.1 M tokens: $26 / $10 / $26. **Written-protocol monthly cost: roughly $160 / $117 / $563.** The written protocol is a liability about ten times the practised cost; current Claude models under-comply with it, a literal-compliance model would pay it.

3.4 Retry loops (VERIFIED counts; markers partly noisy)

Exact harness marker `PreToolUse:Bash hook error:` with `is_error: true` (second mining pass): **149 denials in 78 sessions**. By reason: deletion blocked (`rm` outside temp roots) 76; destructive git blocked pending preflight 43 to 54; credential echo 15; `git add -A` 6; Co-Authored-By 3; gcloud 3; sudo 1; push to main 1. Strict same-command retries 4; loose retries (same first 40 chars) 111; designed preflight-then-retry sequences 20 (19 after destructive-git denials). Hook rejections of the commit itself: `commit-msg` subject-length 0, `smart-commit` errors 0; `claim-dev-merge` failures 16 (force-push of dev rejected or dirty tree); push rejected 12.

Example (VERIFIED quote, `0d08187b...jsonl:684`): "within 12h. Run first: ~/.claude/hooks/git-preflight.sh "/Users/waylon/local_stage/helpen-mpfix/helpen.ms" "before git branch -D" then retry the exact command." A destructive-git denial costs one preflight plus one retry (the deny text carries the fix), about 1,000 to 1,500 new tokens plus one extra turn of cache read (about 91 K tokens, $0.023 on Fable): the 20 designed sequences per month cost under $1. The 76 `rm` denials are CLAUDE.md rule 12 working, not a git ceremony cost. The 16 claim failures are the one git loop with substance (dirty tree, dev force-push refused) and map to change 10. The brief's loose string markers over-count 2 to 50x (first-pass table in results.md section 5: "denied" 319, "claim failure" 808, mostly skill prose, listings and OS errors); the first pass is kept there with quoted examples so the over-count is auditable.

3.5 Where the re-reads come from

Re-reads (same path more than once) mean 1 per session, max 11 (`168e6c49.../agent-a8f5a22ae2eaeea4c.jsonl`: 16 surface reads of 5 files). Causes visible in the receipts: slicing one file with `sed -n` ranges in several calls (two to three reads for one skill), compaction followed by re-loading, and the canonical-copy ambiguity (reads of both `/Users/waylon/local_stage/how-git/...` and `/private/tmp/claude-502/audit-v4rlib/.io.v4rian.how-git/...` copies in `agent-af4da567f34d01aa9.jsonl`).

4. Changes that cut tokens with enforcement and DLP unchanged

Macro untouched: atomic commits, pool/safety pairs, documented history, human-readable commits as the enforcement control, preflight refs, bak zips, safety branches, history preservation. Every change below leaves the hooks, rulesets, allocator, claim scripts, CI gates and the judge lens in place; only what a model reads changes.

Baseline per invocation: 65,000 tokens mandated (section 1). Frequencies and $ per month come from section 3.

#	Change	Files	Enforcement/DLP still owned by	Tokens saved per task	Risk	Savings/risk
1	**Task-indexed loading.** Loader step 2 becomes: "read the skills the decision-tree row for this task names; nothing else"; delete "read every git-* skill" and "read [Git_Strategy and RUNBOOK] directly". Keep "Rules at a glance" as the cross-cutting floor.	`~/.claude/skills/how-git/SKILL.md`:13	guard-destructive.sh, githooks, rulesets, allocator, required.* CI gates, judge lens (all unchanged)	commit 61,000; branch 62,000; merge-to-dev 61,000; merge-to-main 57,000; rewrite 60,000; rebuild 0 to 35,000	Low: a rule in an unloaded skill is reachable by the decision tree; the rules that must hold without reading are hooked	1
2	**Split** `git-commit-audit` into the compose lens (A, B.7, C.11-14, D, J; ~6 KB) and a rebuild appendix (F, I, V.10 recipes) loaded only by git-snapshots/git-shuttle; delete the STATUS paragraph, the dead links, rules 10/15/17/28-30 (hooked), fix A.5/G.36/H.40-41 contradictions.	`.io.v4rian.how-git/skills/git-commit-audit/SKILL.md` (and fractaris copy)	prepare-commit-msg, commit-msg, pre-commit, guard-destructive.sh; the CI judge still receives the (now shorter and consistent) lens	1,800 per commit read; also 1,800 per push in CI (`claude_judge_commit_range` embeds the lens)	Low: the removed items are hook-enforced; the contradictions removed were net negative	2
3	**Loader diet.** Remove rules at a glance that hooks or rulesets own (line 23 push-main: guard; 31 trash safety branches: `safety-lockdown.json`; 34 dash scrub: pre-commit; 33 and 36-37 off-grid workarounds: move to RUNBOOK or into the tools per #10); move "Lessons from 2026-10-28" into the skills they belong to or memory (`feedback_pool_branch_size` already holds one); fix the canonical path (line 13 fractaris versus line 81 how-git).	loader	unchanged	1,700 per invocation (11.5 KB to ~5.5 KB)	Low	3
4	**Prune dead** `[[links]]` **and absent-workflow STATUS paragraphs** in all 13 skills; extend `skill-lint.sh` FW to lint the vendored tree so they stay dead-link free.	all `skills/*/SKILL.md`; `~/.claude/hooks/skill-lint.sh`	unchanged	~1,200 per invocation plus 1 to 3 avoided failed Read/grep round trips (300-600 each)	None	4
5	**Replace RUNBOOK/Git_Strategy full reads with CLI help.** The decision tree points at `bin/status`, `bin/create-pool-branch --help`, `bin/init-pool-merge --help`, `bin/ship --help` (CLAUDE.md rule 7 "use the project tooling"); the two docs become reference, read on a named question.	loader decision tree; no doc edits	unchanged; the bins are the ceremony	19,900 per invocation (already counted inside #1; standalone value if #1 is rejected)	Low: `bin/*` help must state the ceremony; RUNBOOK lines 122-180 already do	5
6	`git-shuttle` **cut** from 42 KB to ~12 KB: drop the voice-ask/Meta-rule 7 copy (CLAUDE.md rule 4 owns it), the per-prompt scripts, the nonexistent `tools/*.sh`, `[CLEANUP]`, `feature/*`, "Changes: always", LA-timezone; point at `bin/reconstruct` once `dev-landing` lands (section 6).	`skills/git-shuttle/SKILL.md`	DLP unchanged: preflight (`git-preflight.sh`), mirror clone, safety branches stay as steps	8,400 per rebuild task; 8,400 per every invocation until #1 lands	Low-medium: rebuild tasks are the one place prose detail earns its tokens; keep the methodology, cut the UI scripting	6
7	**Rename in-repo** `how-dev` **to** `how-ci-watch` and gate it on `MODE=connected`; `how-push` stays the default.	`skills/how-dev/`, loader inventory	unchanged	1,560 per invocation in v4rian repos; removes a binding contradiction	None	7
8	**Move** `git-compile-audit/git-runtime-audit` **recipes into** `required.compile.sh`/`required.runtime.sh` **help text**; the skills keep the matrix and the pass criteria (~3 KB each).	two skills; `ci/audits/*.sh`	the required.* gates (unchanged)	4,000 per invocation when loaded (merge-to-main, rebuilds)	Low	8
9	**Deterministic merge-body lint**: move `git-merge-audit`:141-142 regexes into `audit_message` for `[MERGE]` subjects; collapse the three body-shape rules to one (nested list, per Joel's last correction, `git-merge-audit`:43).	`scripts/lib/auto-fix-commit-message.sh`, `git-merge-audit`, `git-commit-audit` H	stronger: a hook now rejects what only prose rejected	~900 per merge; removes a contradiction the judge is currently asked to resolve	Low	9
10	**Dirty-tree claim and allocator numbering as tool behaviour**: `bin/finalize-pool-merge` stashes untracked files itself; `create-pool-branch` consults `git ls-remote` (pool n165 does part of this). Delete loader:33, 37, 83 once landed.	`bin/*`, `scripts/allocate-pool-branch.sh`, loader	stronger	~600 per invocation plus the retry loops in section 3	Low	10

Monthly value at the observed frequency (October 2026: 32 loader invocations, 100 how-git sessions, 707 commits, 269 merges, 260 pushes, about 150 turns per session)

Two baselines, because the written protocol and the practised protocol differ by about ten times (section 3.2: measured direct load 6,000 tokens per Tier-A session versus 65,000 mandated). "Occupancy" = tokens x turns remaining in the session x cache-read price; "write" = tokens x 5-minute cache-write price. Tokens at bytes/3.6 (floor; likely 1.3 to 1.8x higher). The "practised" column below was computed in the first pass over 100 Tier-B sessions and 150 turns; the second pass (32 Tier-A sessions, 258 later turns) scales each practised figure by about 0.55, so read that column as an upper bound and section 5 as the corrected total.

#	Change	Tokens saved per task	Per month, written-protocol baseline (Fable / Opus 5.5 / Astra)	Per month, practised baseline (Fable / Opus 5.5 / Astra)
1	Task-indexed loading	57,000 to 62,000 per invocation (commit, branch, merge, rewrite); 0 to 35,000 per rebuild	32 x 60,000 = 1.9 M write + 288 M occupancy: **$96 / $67 / $312**	the loader already goes unobeyed; the saving is the 10 KB of surplus reads per session (mean 22.8 KB read vs about 13 KB needed): 2,700 tokens x 100 = 0.27 M write + 40 M occupancy: **$13 / $9 / $43**, plus removing the liability column to the left for any literal-compliance model
2	Split and trim `git-commit-audit` (12.7 KB to about 6 KB)	1,900 per read; 1,900 per push in the CI judge prompt	counted in #1 for the model; CI judge: 260 pushes x 1,900 = 0.5 M uncached input: **$5 / $2 / $5**	model: 100 sessions x 1,900 = 0.19 M write + 28 M occupancy: **$9 / $7 / $31**; CI judge as left: total **$14 / $9 / $36**
3	Loader diet (11.5 KB to about 5.5 KB)	1,700 to 3,000 per invocation	32 x 2,300 = 74 K write + 11 M occupancy: **$3.7 / $2.6 / $12**	same (the injection is always paid): **$3.7 / $2.6 / $12**
4	Dead links and absent-workflow STATUS paragraphs	about 1,200 per loaded skill set plus 1 to 3 avoided failed reads (300 to 600 each)	counted in #1	100 x 2,000 = 0.2 M write + 18 M occupancy: **$7 / $5 / $22**
5	CLI help instead of RUNBOOK/Git_Strategy reads	19,900 per invocation	subset of #1	rarely read today (two sessions): about **$2 / $1.5 / $6**
6	`git-shuttle` 42 KB to about 12 KB	8,400 per rebuild load	32 x 8,400 = 0.27 M write + 40 M occupancy: **$13 / $9 / $43**	about 5 rebuild sessions per month: 42 K write + 6 M occupancy: **$2 / $1.5 / $7**
7	Rename in-repo `how-dev`	1,560 per v4rian invocation	32 x 1,560 = 50 K write + 7.5 M occupancy: **$2.5 / $1.8 / $8**	under $1; value is the removed contradiction
8	Compile/runtime recipes into the required.* scripts	4,000 when loaded	32 x 4,000 = 0.13 M write + 19 M occupancy: **$6 / $4.5 / $21**	about 20 loads: **$4 / $3 / $13**
9	Deterministic merge-body lint	900 per merge read	small (about **$2** on Fable)	small; value is one body shape instead of three
10	Dirty-tree claim and allocator numbering as tool behaviour	about 600 per invocation plus the loops	**$1**	**$1**; no marker for these loops this month

Items 1 and 5 overlap (5 is a subset of 1). Items 2, 3, 4, 6, 7, 8 are additive to 1 only for the skills still loaded by the decision tree; their full value stands alone if 1 is rejected.

Adversarial check on each proposal

- *Does lazy loading let a model skip an un-hooked rule?* The un-hooked rules are judgment rules: atomicity, `[NEW]` vs `[REFACTOR]`, bullet discipline, receipts, user-authored-commits-read-only, merge body tone, pool size. After change 1 they live in `git-commit-audit` (loaded on every commit), `git-merge-audit` (every merge), `git-history-preservation` (every rewrite) and the loader floor, which is exactly where the decision tree sends the model. The one rule that spans tasks without a hook, "never touch Joel's commits", is CLAUDE.md:17 (always loaded) and `guard-destructive.sh` blocks the mechanics (rebase, filter-branch, force) until a preflight exists. Residual risk: an unusual task (cherry-pick across repos) whose row is missing; mitigation is a decision-tree row for `git-reconcile-intent`, which exists (loader:58 inventory but no tree row: add it).
- *Does trimming `git-commit-audit` weaken the CI judge?* The judge receives the lens body verbatim. Removing hook-enforced items removes nothing the judge could catch that the hook does not already reject before the push. Removing contradictions (`cat VERSION`, `feature/`, "straight to dev") removes false FAIL verdicts.
- *Does cutting RUNBOOK reads lose a ceremony step?* The ceremony is executed by `bin/*` and `scripts/*`, not by the reader; a model that runs `bin/init-pool-merge` cannot skip the claim. The off-grid workarounds (dirty tree, allocator numbering) are the exception and belong in the tool (#10) rather than in 11 KB of prose.
- *Could a leaner loader make a model skip the preflight?* No: `guard-destructive.sh` denies the destructive command and prints the preflight command; the loop is one retry.
- *Does removing duplicated statements reduce "human readability as the enforcement control"?* The commits are the readable artefact, not the skills; the lens that shapes them gets shorter and self-consistent.

5. Totals

Per typical task, tokens of git-protocol prose entering context (bytes/3.6 floor; multiply by 1.3 to 1.8 for the likely value per the 3.3 calibration):

Task	Written protocol today	Practised today (median session, section 3.2)	After all changes	Reduction vs written	Reduction vs practised
Compose a commit	65,000 (+5,400 Read prefixes)	about 13,000 (loader 3,200 to 5,400 + `git-commit-audit` 3,500 + memory 400 + one re-read about 3,500)	about 4,100 (loader 1,500 + trimmed lens 1,700 + memory 400 + no re-read)	94%	68%

Task	Written protocol today	Practised today (median session, section 3.2)	After all changes	Reduction vs written	Reduction vs practised
Create a working branch	65,000	about 9,000 (loader + `git-strategy-audit` slice)	about 2,300 (loader + `create-pool-branch --help`)	96%	74%
Merge working branch to dev	65,000	about 11,000 (loader + `git-merge-audit` + RUNBOOK slice)	about 4,500 (loader + trimmed merge lens + `bin/init-pool-merge --help`)	93%	59%
Merge dev to main	65,000	about 15,000 (adds runtime-audit slice)	about 5,500 (plus `required.runtime.sh` help)	92%	63%
History rewrite	65,000	about 12,000 (how-preflight + preservation + rewrite)	about 7,000 (same three, dead links and duplicated preflight prose removed)	89%	42%
Snapshot rebuild	65,000 mandated, about 110,000 if the sub-skills are read as instructed	about 40,000	about 25,000 (shuttle cut, appendix split)	62% to 77%	38%

Correction from the second mining pass (section 3.2): the measured direct load of a Tier-A session is 23.1 KB, about 6,000 tokens mean (max 13,800), lower than the 13,000 to 25,000 "practised" column above, which was built from the wider Tier-B set. Using the measured figure: practised per session about 6,000 tokens; after the changes about 4,000 (loader 1,500 + one trimmed lens 1,700 to 2,000 + memory 400 to 800); **reduction about 30 to 35 percent against measured practice, about 93 percent against the written protocol.** Monthly dollars at list prices: practised about $15 (Fable) / $11 (Opus 5.5) / $50 (Astra) falls to about $10 / $7 / $33; written-protocol compliance about $160 / $117 / $563 falls to the same after-figure. The CI judge (`required.commit-audit`) saves a further 0.5 M uncached input tokens per month from change 2 alone ($5 Fable-priced, $2 Opus-priced).

What this means for the decision: in dollars the practised ceremony is already small (0.18 percent of the month's new tokens); the material savings are (a) removing the ten-times-larger obligation the written loader imposes on any model that obeys it, (b) the quality cost of eight contradictions and 41 dead links inside the lens that the model and the CI judge read on every commit, and (c) the CI judge's per-push prompt. The ranking in section 4 does not depend on the dollar figures.

Confidence: byte counts and file:line receipts HIGH (VERIFIED); frequencies HIGH for October 2026 only (effectively 2.5 live days, 27 to 29 October, plus one June remnant; subagent-heavy: 25 of 32 Tier-A sessions); token-per-byte MEDIUM (bounded 2.0 to 3.6 by one measured turn type, no `count_tokens` credential available: `ant` is not installed and `ANTHROPIC_API_KEY` is unset, VERIFIED); dollar totals LOW-MEDIUM (occupancy uses about 258 later turns per session and a 1.5 M-token upper-bound attribution).

6. Unmerged branch verdict (VERIFIED: `git log dev..<b>`, `git diff dev...<b> --stat`)

- `dev-landing` (71e6173, 2026-09-13) = `landing-base` (aa1a68c) = `local/reconstruct-tool-pending-allocation` (983ca80, same patch, different SHA): one commit `[NEW] Add the deterministic bin/reconstruct tool for git-reconstruct` plus its merge: `bin/reconstruct` (+104), `scripts/reconstruct.sh` (+534), `skills/git-reconstruct/SKILL.md` (+302 lines), README +8, RUNBOOK +6, CHANGELOG +2, lifecycle tests +287. Effect on conclusions: it **adds** a 14th skill (about 10.9 KB; the user copy at `~/.claude/skills/git-reconstruct/SKILL.md` is 10,852 B) to the "read every git-*" fan-out, raising the mandated load by about 3,000 tokens, while delivering the deterministic tool that lets `git-shuttle` shrink (change 6). It strengthens change 1 (one more skill nobody needs for a commit) and does not weaken any conclusion.
- `pool/v2026.0/n165.fix.off-grid-allocator-ignores-clone-refs` (4 commits, 2026-09-03): `scripts/allocate-pool-branch.sh` +37/-2, lifecycle tests +53: the allocator counts remote-tracking and archived refs and reads spent numbers from history. Effect: it is the tool-side fix for loader:33 and loader:83 (change 10); once merged those two loader paragraphs (about 330 tokens) can go. No other conclusion changes.

7. What could not be verified

- Exact token counts: no `count_tokens` credential (`ant` absent, `ANTHROPIC_API_KEY` unset); bytes/3.6 is a heuristic bounded only by the measured Skill-ingest turn (2.0 to 3.6 bytes per token). All token figures are floors.
- Usage before October 2026: 107 of 108 project directories have no transcripts; "per month" is one month.
- The 15 to 25 percent occupancy share of git prose in each turn's cache read: computed from loaded bytes against the mean 91 K cached context, not from per-token attribution.
- Whether the loader's "git rules loaded (memories + skills ...)" confirmation lines matched the files actually read (not sampled).
- GPT-6 Astra's instruction-following behaviour on contradictory rule sets; only its published context, pricing and cache numbers are verified. The openai.com announcement page returned 403; the developer docs pages were fetched.
- The hook-rejection markers for `commit-msg`, `commit-audit`, `smart-commit` and `claim`: the regexes matched prose and listings, so the absence of a measured retry loop there is "not demonstrated", not "shown absent".
- The dirty-tree claim refusal and allocator renumbering loops (loader lines 37, 83): documented in the loader from the 2026-10-28 rebuilds but no distinct marker in the retained transcripts.
- `fractaris/.io.v4rian.how-git` as the canonical copy: it diverges from how-git dev in `git-commit-audit` (97 diff lines); which side is newer was not established (the fractaris tree carries no `FRAMEWORK_REPO` marker and `git log` was not run there to keep the audit inside the stated read-only scope).